

Dynamic Bonding Curve

0.1.6

Smart Contract Security Assessment

October 2025

Prepared for:

Meteora

Prepared by:

Offside Labs

Ronny Xing

Sirius Xie





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	6
4	Key Findings and Recommendations	7
4.1	Incorrect Rounding Direction in <code>get_next_sqrt_price_from_output</code>	7
4.2	Lack of Constraint for <code>MeteoraDammMigrationMetadata</code> Account	8
4.3	Fee Reduction via Input Splitting In <code>RateLimiter</code> Mode	9
4.4	Missing Validation on <code>FeeRateLimiter</code>	10
4.5	Informational and Undetermined Issues	12
5	Disclaimer	14



1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers, operating systems, IoT devices, and hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple, Google, and Microsoft*, have protected digital assets valued at over **\$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **\$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



<https://offside.io/>



<https://github.com/offsidelabs>



https://twitter.com/offside_labs



2 Executive Summary

Introduction

Offside Labs completed a security audit of *Dynamic Bonding Curve* smart contracts, starting on May 12th, 2025, and concluding on August 14th, 2025.

Project Overview

The Dynamic Bonding Curve (DBC) is a permissionless launch pool protocol that enables users to create and launch tokens with customizable bonding curves directly. The program allows partners to configure key parameters such as quote tokens, token graduation curves, and fees.

In the current release, the protocol now supports the two new swap modes ExactOut and PartialFill, and an innovative base fee mode Rate Limiter has been implemented with related trading restrictions. Creators and partners have greater flexibility to mint the base token, withdrawal of the migration fee is supported.

Audit Scope

The assessment scope contains mainly the smart contracts of the dynamic-bonding-curve program for the *Dynamic Bonding Curve* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- Dynamic Bonding Curve
 - Codebase: <https://github.com/MeteoraAg/dynamic-bonding-curve>
 - Commit Hash: 92cc243be4a1f1bb7eaadda24c54b7db5fca941b
 - PR-75
 - Commit Hash: f1ad7034bf21a7672b3270154af869aed91046f8
 - Codebase Link: [PR-75](#)
 - PR-78
 - Commit Hash: 39d7c7fae56c363fe7fd3bd6f24659770bfe6a00
 - Codebase Link: [PR-78](#)
 - PR-83
 - Commit Hash: 3bf57ac1800938afb32c3fbf206aa6a49ec2e3a3
 - Codebase Link: [PR-83](#)
 - PR-84
 - Commit Hash: 1b6339648c98d9b3d7e6132c2f5343818f76e5f9
 - Codebase Link: [PR-84](#)
 - PR-86
 - Commit Hash: b13ed38ca14957aaf28aeeda0a611ed9c2915959
 - Codebase Link: [PR-86](#)
 - PR-113
 - Commit Hash: 73fffa05d93df38b7e079e422c98addd6f1ce8e8



- Codebase Link: [PR-113](#)
- PR-124
 - Commit Hash: a5c710bfec8f45b03937fc4091f7a3dd6707b7a9
 - Codebase Link: [PR-124](#)

We listed the files we have audited below:

- Dynamic Bonding Curve
 - programs/dynamic-bonding-curve/src/base_fee/*.rs
 - programs/dynamic-bonding-curve/src/const_pda.rs
 - programs/dynamic-bonding-curve/src/constants.rs
 - programs/dynamic-bonding-curve/src/curve.rs
 - programs/dynamic-bonding-curve/src/error.rs
 - programs/dynamic-bonding-curve/src/event.rs
 - programs/dynamic-bonding-curve/src/instructions/admin/ix_claim_pool_creation_fee.rs
 - programs/dynamic-bonding-curve/src/instructions/admin/ix_withdraw_lamports_from_authority.rs
 - programs/dynamic-bonding-curve/src/instructions/admin/ix_withdraw_protocol_surplus.rs
 - programs/dynamic-bonding-curve/src/instructions/creator/ix_transfer_pool_creator.rs
 - programs/dynamic-bonding-curve/src/instructions/initialize_pool/ix_initialize_virtual_pool_with_spl_token.rs
 - programs/dynamic-bonding-curve/src/instructions/initialize_pool/ix_initialize_virtual_pool_with_token2022.rs
 - programs/dynamic-bonding-curve/src/instructions/initialize_pool/process_create_token_metadata.rs
 - programs/dynamic-bonding-curve/src/instructions/ix_swap.rs
 - programs/dynamic-bonding-curve/src/instructions/migration/create_locker.rs
 - programs/dynamic-bonding-curve/src/instructions/migration/dynamic_amm_v2/*.rs
 - programs/dynamic-bonding-curve/src/instructions/migration/flash_rent.rs
 - programs/dynamic-bonding-curve/src/instructions/migration/ix_withdraw_migration_fee.rs
 - programs/dynamic-bonding-curve/src/instructions/migration/meteora_damm/*.rs
 - programs/dynamic-bonding-curve/src/instructions/migration/mod.rs
 - programs/dynamic-bonding-curve/src/instructions/partner/ix_create_config.rs
 - programs/dynamic-bonding-curve/src/instructions/swap/*.rs
 - programs/dynamic-bonding-curve/src/lib.rs
 - programs/dynamic-bonding-curve/src/math/safe_math.rs
 - programs/dynamic-bonding-curve/src/math/utils_math.rs
 - programs/dynamic-bonding-curve/src/params/fee_parameters.rs
 - programs/dynamic-bonding-curve/src/params/liquidity_distribution.rs
 - programs/dynamic-bonding-curve/src/state/config.rs
 - programs/dynamic-bonding-curve/src/state/fee.rs



- `programs/dynamic-bonding-curve/src/state/virtual_pool.rs`

Findings

The security audit revealed:

- 0 critical issue
- 1 high issue
- 1 medium issue
- 2 low issues
- 2 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Incorrect Rounding Direction in get_next_sqrt_price_from_output	High	Fixed
02	Lack of Constraint for MeteoradammMigrationMetadata Account	Medium	Fixed
03	Fee Reduction via Input Splitting In RateLimiter Mode	Low	Fixed
04	Missing Validation on FeeRateLimiter	Low	Acknowledged
05	Two Rounding-Ups May Lead to Larger Base Fee	Informational	Fixed
06	VestingEscrow's Recipient Cannot Be Transferred	Informational	Fixed



4 Key Findings and Recommendations

4.1 Incorrect Rounding Direction in `get_next_sqrt_price_from_output`

Severity: High

Status: Fixed

Target: Smart Contract

Category: Precision

Description

The `get_next_sqrt_price_from_output` choose the incorrect rounding direction when calculating for an exact amount of tokens out. It rounds in a way that benefits the user, not the protocol.

```
179     pub fn get_next_sqrt_price_from_output(  
180         ...  
181     ) -> Result<u128> {  
182         ...  
183         if base_for_quote {  
184             get_next_sqrt_price_from_amount_quote_rounding_up(sqrt_price,  
185             ↪ liquidity, amount_out)  
186         } else {  
187             ↪ get_next_sqrt_price_from_amount_base_rounding_down(sqrt_price,  
188             ↪ liquidity, amount_out)  
189         }  
190     }
```

[programs/dynamic-bonding-curve/src/curve.rs#L179-L188](#)

For an exact output swap, the protocol must always round the price in a way that makes the user pay slightly more input tokens. This protects the pool's liquidity. The current implementation does the exact opposite, making the user pay slightly *less*.

- When the price should go down, it rounds it up (closer to the original price).
- When the price should go up, it rounds it down (closer to the original price).

Impact

The protocol undercharges users, it pays out more than it takes in. Attackers can repeatedly use this flaw to drain the base/quote token from pool.

Recommendation

It's recommended to reverse the rounding directions in `get_next_sqrt_price_from_output` :



- When `base_for_quote == true` , it should use `get_next_sqrt_price_from_amount_quote_rounding_down` .
- When `base_for_quote == false` , it should use `get_next_sqrt_price_from_amount_base_rounding_up` .

Mitigation Review Log

Fixed in the commit `be3d8eed0075a40c1c8375a7cfde024d81efad34`.

4.2 Lack of Constraint for MeteoraDammMigrationMetadata Account

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Data Validation

Description

The `transfer_pool_creator` instruction loads the `MeteoraDammMigrationMetadata` account from `remaining_accounts` if the `migration_option` is `MigrationOption::MeteoraDamm` .

The issue is that, it doesn't check if the loaded `MeteoraDammMigrationMetadata` has same `virtual_pool` with the current context.

```
46     let migration_metadata_account = ctx
47         .remaining_accounts
48         .get(0)
49         .ok_or_else(|| PoolError::InvalidAccount)?;
50     let migration_metadata_loader: AccountLoader<'_,
51         MeteoraDammMigrationMetadata> =
52         AccountLoader::try_from(migration_metadata_account)?;
53     let migration_metadata = migration_metadata_loader.load()?;
```

[programs/dynamic-bonding-curve/src/instructions/creator/ix_transfer_pool_creator.rs#L46-L52](#)

Impact

The previous creator can input an incorrect `MeteoraDammMigrationMetadata` account from other `virtual_pool` to bypass the `migration_metadata` status check.

Although, this could only corrupt their own intermediate state, without gaining any additional benefits.



Recommendation

Add `virtual_pool` constraint for the `MeteoraDammMigrationMetadata` account.

Mitigation Review Log

Fixed in the commit 9666b940ce93e79dd9f7c5da5ba6c84bb84620f5.

4.3 Fee Reduction via Input Splitting In RateLimiter Mode

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

When the Virtual Pool's `BaseFeeMode` is set to `RateLimiter`, any portion of the `input_amount` exceeding `FeeRateLimiter.reference_amount` will be subject to a higher fee rate than `FeeRateLimiter.cliff_fee_numerator` during the interval `[activation_point, activation_point + FeeRateLimiter.max_limiter_duration]`.

However, users can bypass these higher fees by splitting their `input_amount` into multiple smaller swaps, each no larger than `FeeRateLimiter.reference_amount`.

Impact

If Dynamic Fee is not enabled, this method allows users to easily avoid paying high fees, effectively capping their fee rate at `FeeRateLimiter.cliff_fee_numerator` through several micro-swaps.

Even with Dynamic Fee enabled, users might still reduce their total fees through careful calculations and strategic splitting, depending on the configurations of `FeeRateLimiter` and `DynamicFeeConfig`. For instance, splitting a large `input_amount` into two swaps of `input_amount / 2` can reduce the Base Fee, and since this approach triggers fewer Dynamic Fee adjustments, it may further lower the total fees paid.

Recommendation

It is recommended to require Dynamic Fee to be enabled whenever the Virtual Pool's `BaseFeeMode` is set to `RateLimiter`. This helps prevent users from avoiding high base fees through multiple small swap.

Mitigation Review Log

Fixed in the commit a36d575dfb88b1ce29a3ad99e9b24749276b5ca2.



Meteora Team: We can mitigate that by blocking several ix's in 1 tx, but snipers can still use multiple transactions (just a bit harder for them), so another mitigation is to create config key including both rate limiter and dynamic fee as your suggestion, but it will be from integration side.

5/17/2025 Update:

Meteora Team: Add more validation to prevent multiple swaps in CPI [PR-79](#)

Offside Labs: The mitigation can limit strictly that there is only one swap instruction invocation in one transaction.

But need to note that, this strict validation will also diminish composability and block certain integrations:

1. The following check for the pool account with mutable flag `instruction.accounts[i].is_writable` is always true:

```
316 // we treat any instruction including that pool address is other swap ix
317 for i in 0..instruction.accounts.len() {
318     if instruction.accounts[i].pubkey.eq(pool) &&
319         instruction.accounts[i].is_writable {
319         msg!("Multiple swaps not allowed");
320         return Err(PoolError::FailToValidateSingleSwapInstruction.into());
321     }
322 }
```

[programs/dynamic-bonding-curve/src/instructions/ix_swap.rs#L316-L322](#)

Because transaction-level instructions (which operate at stack height 1) cannot control the flags in the AccountMeta list and can only adhere to the flags specified in the transaction accounts list. Additionally, since the pool account must be mutable for swap instructions, the pool account is always marked as writable for all transaction-level instructions.

So this check for the transaction level instructions, `instruction.accounts[i].is_writable`, will always be true.

2. `validate_single_swap_instruction` also checks the CPI stack height will never greater than 2. This might block certain integration from Jupiter, for example Jupiter Limit Order with Jupiter Swap API V6 which invokes swap of DEX programs at the stack height of 3.

4.4 Missing Validation on FeeRateLimiter

Severity: Low

Status: Acknowledged

Target: Smart Contract

Category: Data Validation



Description

The `FeeRateLimiter.validate` function currently lacks sufficient checks and constraints on parameters such as `fee_increment_bps` and `reference_amount`. This can lead to abnormal configurations, resulting in the following potential issues:

- When `fee_increment_bps` is set too high, it may cause `get_max_index` to return 0, which leads to the portion of `input_amount` exceeding `reference_amount` being charged directly at the maximum fee rate (`MAX_FEE_NUMERATOR`).
- When `reference_amount` is extremely small (e.g., 1), the computed value of `a` becomes large, causing a significant portion of the `input_amount` to be charged at `MAX_FEE_NUMERATOR`.
- When both `cliff_fee_numerator` and `fee_increment_bps` are set to very small values, it may result in an excessively large `max_index` (up to 9899). This can further cause the calculated `numerator_1` to exceed `FEE_DENOMINATOR`.

Impact

Abnormal `FeeRateLimiter` configurations can lead to unexpected intermediate values during fee numerator calculations, making it highly likely for the final `base_fee_numerator` to hit the maximum limit `MAX_FEE_NUMERATOR`.

Proof of Concept

```
FeeRateLimiter:
  cliff_fee_numerator: 10_000
  fee_increment_bps: 1
  reference_amount: 100

input_amount: 990100

Output:
  a: 9900  b: 0
  max_index: 9899
  numerator_1: 4900995000000 numerator_2: 990000000
  first_fee: 490100 x0: 100
  second_fee: 99 left_amount: 100
  trading fee: 490199
  base_fee_numerator: 495100495
```

With certain parameter combinations, `numerator_1` can reach 4,900,995,000,000, far exceeding the defined `MAX_FEE_NUMERATOR` of 990,000,000.

Recommendation

It is recommended to introduce additional parameter validations in `get_base_fee_numerator` to prevent such anomalies.



Mitigation Review Log

Meteora Team: Ack, because partner will be one to create config key, so we don't want to block for any kind of configuration, they can easily test a new parameters if old configs won't work.

4.5 Informational and Undetermined Issues

Two Rounding-Ups May Lead to Larger Base Fee

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Precision

When calculating `trading_fee`, the formula is split into two parts, and both parts remove the denominator and apply rounding up separately.

```
73     let trading_fee: u64 = if a < max_index {
74         let numerator_1 = c + c * a + i * a * (a + one) / two;
75         let numerator_2 = c + i * (a + one);
76         let first_fee = (x0 * numerator_1 + denominator - one) /
77             denominator;
78         let second_fee = (b * numerator_2 + denominator - one) /
79             denominator;
80         (first_fee + second_fee)
81         .try_into()
82         .map_err(|_| PoolError::TypeCastFailed)?
83     } else {
```

[programs/dynamic-bonding-curve/src/base_fee/fee_rate_limiter.rs#L73-L81](#)

This double rounding up may cause the calculated `trading_fee` to be larger than the actual value by 1, which in turn leads to the `fee_numerator` returned by `get_fee_numerator_from_amount` being larger by 1 as well.

It is recommended to adjust the calculation so that the rounding up is handled after summing the two parts.

$$\frac{x0 * numerator_1 + b * numerator_2 + denominator - one}{denominator}$$

Since the calculation here uses the `U256` type, there is no risk of overflow.

A similar issue also exists in this `else` branch:



```
81     } else {
82         let numerator_1 = c + c * max_index + i * max_index * (max_index +
            ↪ one) / two;
83         let numerator_2 = U256::from(MAX_FEE_NUMERATOR);
84         let first_fee = (x0 * numerator_1 + denominator - one) /
            ↪ denominator;
85
86         let d = a - max_index;
87         let left_amount = d * x0 + b;
88         let second_fee = (left_amount * numerator_2 + denominator - one) /
            ↪ denominator;
89         (first_fee + second_fee)
90         .try_into()
91         .map_err(|_| PoolError::TypeCastFailed)?
```

[programs/dynamic-bonding-curve/src/base_fee/fee_rate_limiter.rs#L81-L91](#)

Fixed in the commit 53c89db3a6f401c9e94bae8bbba4976744afea2b.

VestingEscrow's Recipient Cannot Be Transferred

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Logic Error

The `create_locker` instruction uses following parameters to create a VestingEscrow for the pool creator:

```
115     Ok(CreateVestingEscrowParameters {
116     ...
117         update_recipient_mode: 1, // only creator
118         cancel_mode: 1,           // only creator
119     })
```

[programs/dynamic-bonding-curve/src/instructions/partner/ix_create_-
config.rs#L115-L119](#)

Because the `update_recipient_mode` is 1, which means only the VestingEscrow creator (`pool_authority`) can update the `recipient` .

So even the pool creator is changed, the recipient of the VestingEscrow can't be updated to the new creator.

Fixed in the commit e958c7cdab0414f1c25f0e86d79e5dbb3f9bea85.



5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

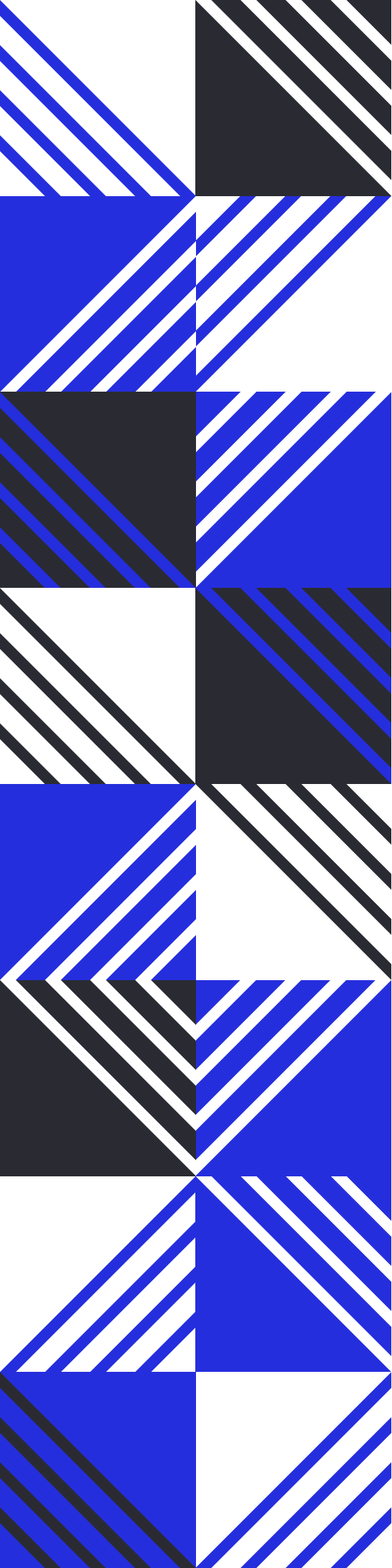
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs